

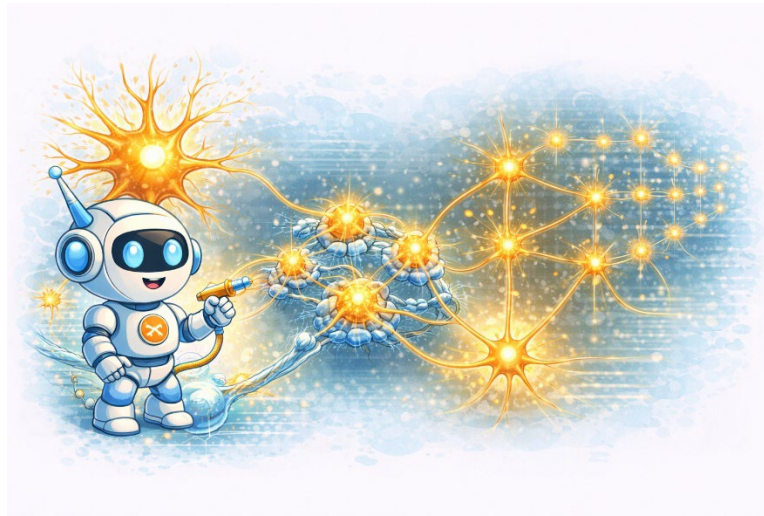
ITAI 4374

Neuroscience as a Model for AI

MODULE 04

Neurons & Spiking Neural Networks

From Biological Spikes to Neuromorphic Hardware



Houston City College

AI & Robotics Program — Bachelor's Degree, Year 4

Spring 2026

Author: Patricia McManus

STUDENT LEARNING OUTCOMES FOR THIS MODULE

SLO 2: Implement simulations of neurons and neural networks using computational tools such as Wolfram System Modeler or snnTorch to demonstrate energy-efficient inference. (Create)

SLO 6: Develop and present a robust, neuro-inspired AI system that addresses a real-world engineering constraint (e.g., power consumption, data scarcity, adaptability). (Create)

Bridge from Module 03 — Where We Are in the Journey 4**The Three Generations of Neural Networks 5**

- 2.1 *First Generation — Binary Threshold Units (1940s–1960s)* 5
- 2.2 *Second Generation — Continuous Activation Functions (1980s–Present)* 5
- 2.3 *Third Generation — Spiking Neural Networks (2000s–Present)* 6

Why Spikes Matter — The Case for Temporal Computation 9

- 3.1 *Energy Efficiency Through Sparsity* 9
- 3.2 *Natural Temporal Processing* 9
- 3.3 *Ultra-Low Latency* 10
- 3.4 *Biological Plausibility — Understanding the Brain* 10

From LIF Neurons to Spiking Neural Networks 11

- 4.1 *Quick Recap: How the LIF Neuron Works (Concepts First)* 11
- 4.2 *Connecting Neurons into a Network* 13
- The Time Loop* 13

Encoding Input Data as Spikes 14

- 5.1 *Rate Coding — More Intense = More Spikes* 15
- 5.2 *Latency Coding — Important Stuff Arrives First* 15
- 5.3 *Delta Modulation — Only React to Change* 15
- 5.4 *Direct Input — Let the Network Figure It Out* 16

The Training Problem — Why Backpropagation Breaks 19

- 6.1 *How Backpropagation Works (Conceptual Review)* 19
- 6.2 *Where It Breaks: The Spike Function* 19

Surrogate Gradients — The Elegant Solution 21

- 7.1 *The Core Trick: Use Two Different Functions* 21
- 7.2 *Common Surrogate Functions* 22
- 7.3 *How This Connects to Biological Learning* 23

Backpropagation Through Time (BPTT) in SNNs 24

- 8.1 *The Concept: Unrolling Through Time* 24
- 8.2 *The Training Loop — Step by Step* 25
- 8.3 *Interpreting Output Spikes — Making a Decision* 25

snnTorch — Your Tool for Building SNNs 27

- 9.1 *Why snnTorch Over Other Options?* 27
- 9.2 *Core Building Blocks* 27

9.3 How a snnTorch Model Looks 27

What Is Neuromorphic Computing? 31

10.1 The Von Neumann Bottleneck 31

Intel Loihi 2 and the Hala Point System 33

11.1 Loihi 2 Architecture 33

11.2 Hala Point — Scaling to a Billion Neurons 33

The Broader Neuromorphic Ecosystem 35

12.1 Other Neuromorphic Platforms 35

12.2 Event-Driven Cameras (Dynamic Vision Sensors) 36

12.3 Real-World Applications Already in Use 37

Lab Preview — Training an SNN on MNIST with snnTorch 39

13.1 What You Will Build 39

13.2 Tools and Setup 39

13.3 What You Will See and Learn 39

Key Takeaways & Bridge to Module 05 40

14.1 What We Learned 40

Comprehensive Review Questions 41

Glossary 43

References & Recommended Resources 44

Core Reading 44

Online Resources 44

Key Papers 45

PART 1

From Single Neurons to Spiking Networks

Building on Module 03's foundation, we connect individual neurons into networks that communicate through spikes.

SECTION 1

Bridge from Module 03 — Where We Are in the Journey



YOUR LEARNING PATH SO FAR

Module 01: WHY — Why study the brain for AI? The energy crisis, historical context.

Module 02: WHAT — What does the brain compute? World models, predictions, theories.

Module 03: WHERE & HOW — Brain anatomy, the single neuron, neural coding.

➡ Module 04: LEARNING & NETWORKS — How do neurons learn together? SNNs, training, hardware.

In Module 03, you built a single LIF neuron. Now we connect thousands of them into networks that can learn.

Think back to Module 03. You learned that a biological neuron is not the simple unit shown in most AI textbooks. It is a dynamic system — it collects incoming signals over time, it leaks energy between inputs (like water slowly draining from a bathtub), and it fires a sudden all-or-nothing spike when it has accumulated enough charge to cross a threshold. After firing, it resets and starts the process again.

You modeled this behavior with the Leaky Integrate-and-Fire (LIF) neuron and saw how the brain encodes information through rate coding (how often neurons fire), temporal coding (when exactly they fire), and population coding (which neurons fire together). These are the raw ingredients. Module 04 is about what happens when you put those ingredients together.

The questions we tackle now are: Can we build entire neural networks from spiking neurons? How do we train them, given that spikes are not smooth functions? And does specialized hardware exist to run these networks efficiently? The answers take us into one of the most exciting frontiers in modern AI.



AI CONNECTION: Why This Module Matters for Your Career

Spiking Neural Networks and neuromorphic computing are not just academic curiosities. Companies like Intel, BrainChip, SynSense, Qualcomm, and Samsung are hiring engineers who understand this technology. The global neuromorphic computing market is projected to grow significantly as industries demand AI that can run on battery-powered edge devices — think drones, medical wearables, autonomous vehicles, and smart sensors.

The skills you build in this module — understanding SNN theory, training models with `snnTorch`, and knowing neuromorphic hardware constraints — are directly applicable to these roles.

SECTION 2

The Three Generations of Neural Networks

The history of neural networks can be organized into three generations, each defined by a different type of computational unit. Understanding this progression explains why spiking neural networks are not just an incremental improvement — they represent a fundamentally different way of computing.

2.1 First Generation — Binary Threshold Units (1940s–1960s)

The journey begins in 1943 when McCulloch and Pitts proposed the first mathematical model of a neuron. Their neuron was simple: add up the inputs, and if the total exceeds a threshold, output 1. Otherwise, output 0. There was no middle ground — the output was strictly binary, like a light switch that is either on or off.

Rosenblatt's Perceptron (1958) built on this idea by adding adjustable weights to the inputs, allowing the network to learn from examples. The Perceptron could learn to separate data into two groups — for example, distinguishing between photos of cats and dogs — but only if the groups could be divided by a straight line. This limitation was famously demonstrated by Minsky and Papert in 1969, who showed that the Perceptron could not solve even simple problems like XOR (where the answer depends on a combination of inputs, not just their sum). This contributed to the first "AI Winter" — a period of reduced funding and interest in neural networks.

2.2 Second Generation — Continuous Activation Functions (1980s–Present)

The breakthrough came when researchers replaced the binary on/off output with smooth, continuous activation functions like sigmoid, tanh, and eventually ReLU. Instead of saying "yes or no," these neurons could say "25% yes" or "97% yes." This seemingly small change had enormous consequences.

With smooth outputs, you could use **backpropagation** — the algorithm that adjusts weights by figuring out how much each weight contributed to the error and nudging it in the right direction. Backpropagation needs smooth, differentiable functions to work (we

will explain exactly why in Section 6). The combination of continuous activations, backpropagation, large datasets, and powerful GPUs produced the deep learning revolution you have studied extensively in your AI coursework.

However, second-generation networks have a fundamental design limitation: they **process information as static snapshots**. An image goes in, a prediction comes out. Even when processing sequences (like in RNNs or Transformers), the neuron itself has no sense of time — it performs one instantaneous computation at each step. There is no internal clock, no accumulation of evidence over time, no "memory" within the neuron.

2.3 Third Generation — Spiking Neural Networks (2000s–Present)

The third generation puts time back into the picture. Spiking neurons — like the LIF model from Module 03 — are tiny dynamical systems. They collect inputs over time, maintain a running tally (the membrane potential), and fire a sharp, discrete spike when that tally crosses a threshold. Then they reset and start again. Information is carried not just by whether a neuron fires, but by when it fires and how often it fires.

This is how your brain actually works. And it opens possibilities that second-generation networks struggle with: extreme energy efficiency (because most neurons are silent most of the time), natural handling of temporal data (because time is built into the neuron), and ultra-low latency (because the network can respond before processing all the data).

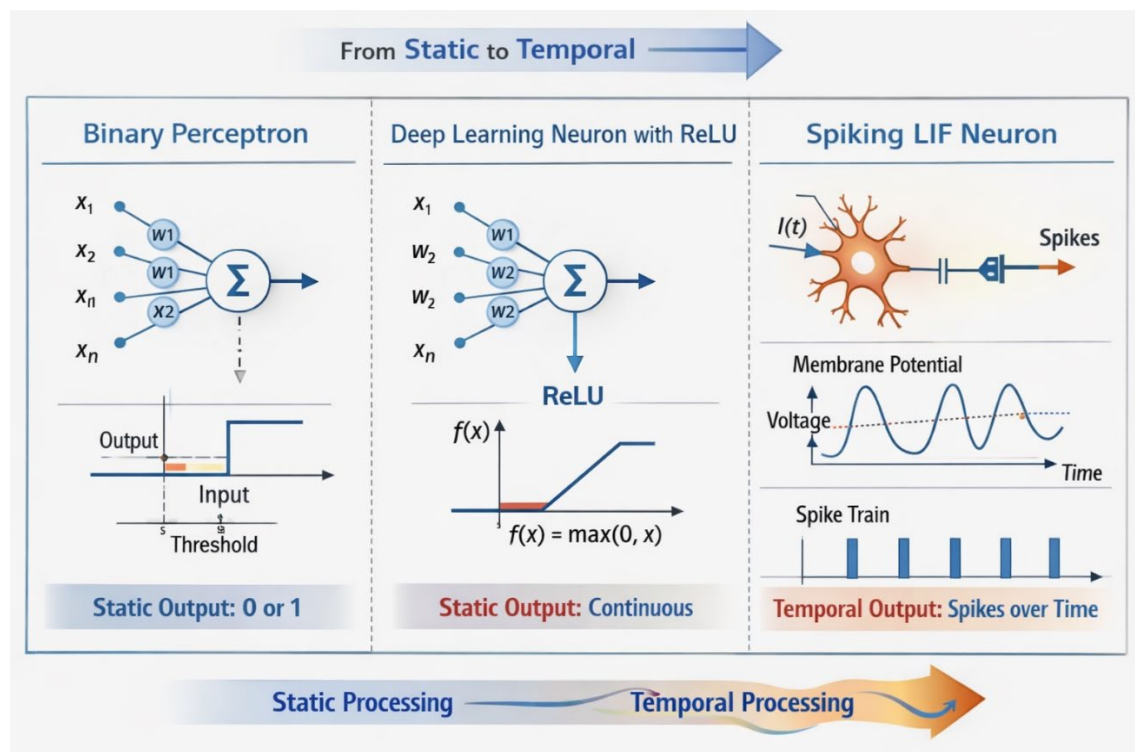
Feature	1st Gen (Perceptrons)	2nd Gen (Deep Learning)	3rd Gen (SNNs)
Neuron output	Binary: 0 or 1	Continuous: 0.0 to 1.0	Discrete spikes in time
Time in the neuron	None	None (implicit in sequences)	Built-in (membrane dynamics)
How it computes	Add inputs → threshold	Add inputs → smooth function	Accumulate over time → spike → reset
How it learns	Perceptron rule	Backpropagation	Surrogate gradients (Sec. 7)
Energy profile	N/A	Dense: every neuron fires every pass	Sparse: most neurons silent most of the time
Best hardware	N/A	GPUs / TPUs	Neuromorphic chips (Sec. 10–12)
Brain similarity	Very low	Abstract inspiration	Closer to biology

⚠ MISCONCEPTION ALERT: "SNNs Are Just a Different Activation Function"

This is one of the most common misunderstandings. It is tempting to think of spiking neurons as simply replacing ReLU with a step function. But that would give you a binary network, not a spiking neural network.

The key difference: a standard neuron has no memory — it computes its output instantly and forgets everything. A spiking neuron is a dynamical system that remembers its past. Its membrane potential carries forward in time, it integrates inputs over multiple time steps, and the timing of its output spike encodes information. The entire paradigm of how data flows through the network is fundamentally different.

Analogy: Think of a standard ANN neuron as a calculator that gives you an answer the moment you press equals. A spiking neuron is more like a pressure gauge that slowly fills, eventually triggers an alarm (spike), and then releases the pressure (reset).



** TRY THIS: Spot the Generations**

Think about the AI systems you have built or studied so far. For each one, identify which generation it belongs to:

- The image classifier you built in Computer Vision — which generation?
- The chatbot powered by a Transformer — which generation?
- The LIF neuron simulation from Module 03 — if you connected many of these, which generation?

Now consider: could any of your previous projects benefit from temporal processing? Think of a scenario where knowing WHEN inputs arrive (not just what they are) would improve performance.

** KNOWLEDGE CHECK**

Test your understanding before moving on:

1. In plain English, what is the key difference between second-generation (deep learning) neurons and third-generation (spiking) neurons?
2. Why was the Perceptron unable to solve the XOR problem, and how did the second generation overcome this?
3. Give two reasons why spiking neural networks could be more energy efficient than conventional deep learning.
4. In your own words, explain the pressure gauge analogy for a spiking neuron. What corresponds to 'filling,' 'triggering the alarm,' and 'releasing the pressure'?

SECTION 3

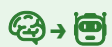
Why Spikes Matter — The Case for Temporal Computation

If second-generation networks are so successful — they power ChatGPT, DALL-E, self-driving cars, and protein folding — why should we care about spikes? The answer lies in four compelling advantages.

3.1 Energy Efficiency Through Sparsity

In a conventional neural network, every neuron in every layer produces an output for every input, every time. Whether the input is a blank white image or a complex scene, the same number of multiplications occur. This is wildly wasteful. Training large language models can cost millions of dollars in electricity, and running AI on battery-powered devices drains batteries quickly.

Spiking neurons flip this model on its head. A neuron only does work when it fires a spike. No spike means **zero computation, zero memory access, zero energy**. In the human brain, at any given millisecond, only about 1–5% of neurons are actively spiking. The rest are silent, consuming virtually no energy. This extreme sparsity is why the brain runs on about 20 watts — roughly the power of a dim light bulb — while processing information with 86 billion neurons.



AI CONNECTION: The Sparsity Principle

Think of it this way: a conventional neural network is like a city where every light in every building is on 24/7, regardless of whether anyone is home. A spiking neural network is like a city where lights turn on only when someone enters a room. The total number of light bulbs (neurons) may be the same, but the energy bill is dramatically different.

Intel's Hala Point neuromorphic system (1.15 billion neurons on Loihi 2 chips) achieves up to 15 TOPS/W (tera-operations per second per watt). Typical GPUs achieve about 1–5 TOPS/W. This is not a small improvement — it is an order of magnitude.

3.2 Natural Temporal Processing

Much of the data in the real world is inherently temporal — it unfolds over time. Audio signals, video streams, sensor readings from robots, radar pulses, and brain signals are all temporal. Standard ANNs handle temporal data by chopping it into fixed-size windows or using attention mechanisms that require storing large context buffers. This works but feels like wearing oven mitts to play piano — you can do it, but you are fighting the tool.

SNNs handle temporal data naturally because time is built into the neuron. The membrane potential serves as a continuously running short-term memory. Spikes carry

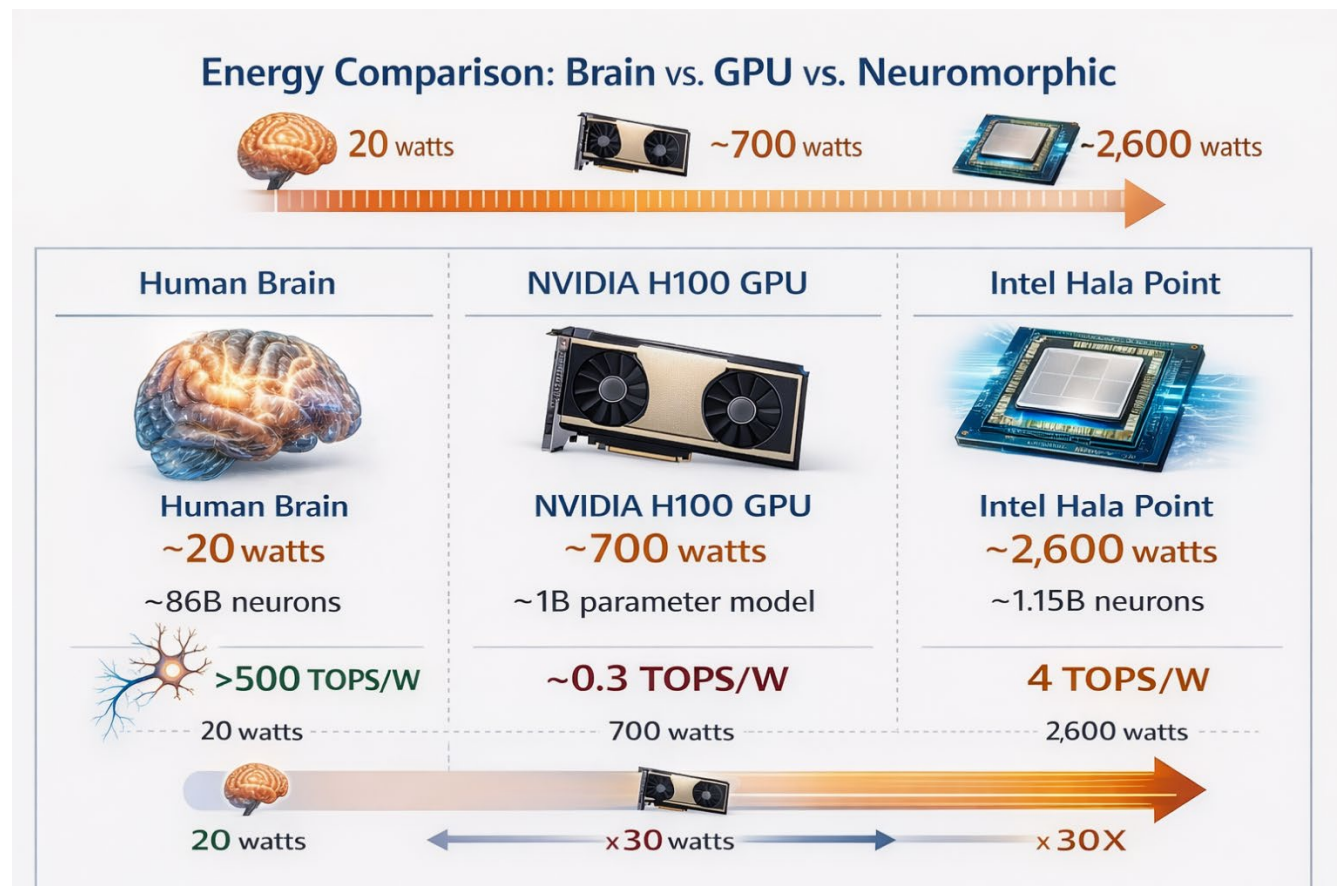
temporal information intrinsically. The network does not need external tricks to process sequences — it is a sequence processor by design.

3.3 Ultra-Low Latency

In a conventional network, you must wait for the entire forward pass to complete before getting an answer. In a spiking network, output neurons can fire as soon as they have accumulated enough evidence — potentially after seeing only a fraction of the input. Imagine a network classifying a clearly written digit '1' — the output neuron might fire after just a few time steps because the evidence is overwhelming, even though the full simulation has not finished. This makes SNNs naturally suited for time-critical applications: autonomous driving, robotic control, real-time monitoring.

3.4 Biological Plausibility — Understanding the Brain

For this course specifically, SNNs matter because they are the closest computational model we have to how actual brains process information. The LIF neurons from Module 03 are the building blocks of SNNs, and studying how these networks learn and process information gives us insight into our own neural circuits. In Module 06, you will study Hebbian learning and Spike-Timing-Dependent Plasticity (STDP) — biological learning rules that only make sense in the context of spiking neurons.



✏ TRY THIS: Energy Cost Estimation

Let's do some back-of-the-envelope math. Suppose a conventional CNN with 10 million neurons processes an image in one forward pass, and every neuron fires (100% activity).

Now suppose an equivalent SNN processes the same image over 25 time steps, but only 3% of neurons fire at any time step.

- How many total neuron activations does the CNN produce? ($10 \text{ million} \times 1 \text{ pass} = ?$)
- How many total spike events does the SNN produce? ($10 \text{ million} \times 3\% \times 25 \text{ steps} = ?$)
- Which number is smaller? By how much?

This is a simplified model, but it illustrates the core efficiency argument. The exact savings depend on hardware — which is why Part 3 matters!

☑ KNOWLEDGE CHECK

Test your understanding before moving on:

1. What does 'event-driven' mean in the context of spiking neurons, and why does it lead to energy savings?
2. Give a real-world example of temporal data that an SNN could process more naturally than a standard ANN.
3. Explain why an SNN could potentially give faster answers than a conventional network, even with the same number of neurons.
4. Using the city lights analogy, describe the difference between how GPUs and neuromorphic chips handle neural activity.

SECTION 4**From LIF Neurons to Spiking Neural Networks**

You already know the LIF neuron from Module 03. Now we need to see how to connect many LIF neurons together into a network that can actually learn to do useful things, like recognize images or process sensor data.

4.1 Quick Recap: How the LIF Neuron Works (Concepts First)

Before we build networks, let us make sure the LIF neuron is crystal clear in conceptual terms. Forget the equations for a moment — here is what the neuron actually does:

Step 1 — Receive input: The neuron receives signals from other neurons or from external data. Each incoming signal is multiplied by a weight (just like in a standard neural network — stronger connections carry more influence).

Step 2 — Integrate: The weighted inputs are added to the neuron's membrane potential — a running tally that accumulates over time. Think of it as a bucket that collects water droplets (inputs).

Step 3 — Leak: Between inputs, the membrane potential slowly decays back toward rest — the bucket has a small hole in the bottom. This leak means the neuron 'forgets' old inputs gradually, unless new inputs arrive fast enough to keep the level up.

Step 4 — Fire (maybe): If the membrane potential crosses the threshold, the neuron fires a spike — a sharp, brief signal sent out to all downstream neurons. This is binary: it either fires or it does not. No half-spikes.

Step 5 — Reset: After firing, the membrane potential resets (the bucket empties), and the process starts again.



4.2 Connecting Neurons into a Network

A spiking neural network has the same layered structure as a conventional neural network — input layer, hidden layers, output layer — connected by weighted links. The difference is what happens inside each node and how information moves through time.

In a conventional network, you feed in one input (say, an image), it passes through all layers in one shot, and you get an output (say, "this is a cat"). The whole thing takes a single forward pass. In a spiking network, the same input is presented across multiple time steps. At each time step, some neurons spike and some do not. These spikes flow to the next layer, where other neurons integrate them. Over many time steps, patterns of activity build up, and the output neurons gradually accumulate evidence for their classification.

Think of it like this: a conventional network makes a snap judgment based on a single glance. A spiking network builds up its decision over time, like a jury hearing evidence from multiple witnesses before reaching a verdict.

The Time Loop

OPTIONAL MATH DETAIL: The LIF Equation (for reference)

If you want to see the formal math, here is the discrete-time LIF update rule:

$$U[t+1] = \beta \times U[t] + W \times X[t+1] - \text{Reset}[t]$$

In plain English: New potential = (decay factor × old potential) + (weighted input) – (reset if we just spiked)

Where β (beta) is the decay factor — a number between 0 and 1 that controls the leak:

- β close to 1 = slow leak = long memory (neuron remembers inputs for a while)
- β close to 0 = fast leak = short memory (neuron forgets inputs quickly)

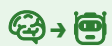
A spike occurs when $U[t] > \text{threshold}$, and the reset term subtracts the threshold to bring the potential back down.

You do not need to memorize this equation. What matters is the concept: integrate, leak, fire, reset.

To simulate an SNN, you run a time loop. For each time step:

1. Convert the input data into spikes for this time step (see Section 5).
2. Each hidden neuron receives weighted spikes from the previous layer and updates its membrane potential.
3. Neurons whose membrane potential crosses the threshold fire spikes.
4. These spikes propagate to the next layer.
5. Output neurons accumulate spikes — after all time steps, the neuron that spiked the most is the predicted class.

A typical simulation runs for 25 to 100 time steps per input. More time steps give the network more time to accumulate evidence but cost more computation.



AI CONNECTION: How `snnTorch` Builds SNNs

In `snnTorch` (the Python library you will use in the lab), you build a spiking network by interleaving standard PyTorch layers with spiking neuron layers:

PyTorch layer (`nn.Linear` or `nn.Conv2d`) → handles the CONNECTIONS between neurons (weights)

`snnTorch` layer (`snn.Leaky`) → handles what happens INSIDE each neuron (integrate, leak, fire, reset)

Everything you know about PyTorch — `DataLoaders`, optimizers, GPU acceleration — works exactly the same way. `snnTorch` just adds the time dimension and spiking behavior.

SECTION 5

Encoding Input Data as Spikes

Before data can enter a spiking neural network, it needs to be converted from conventional formats (like pixel values in an image) into spike trains. This is analogous to how your eyes convert light into electrical signals that your brain can process. The process is called spike encoding, and there are several approaches.

FOUNDATION: Connecting to Your Data Preprocessing Knowledge

In your Deep Learning and Computer Vision courses, you learned that data preprocessing is critical — normalization, augmentation, tokenization. Spike encoding is the SNN equivalent of preprocessing. Just as choosing between different tokenizers affects NLP model performance, choosing between encoding methods affects SNN performance.

The difference: in conventional deep learning, preprocessing is a fixed step before training. In SNNs, encoding converts static data into temporal sequences, adding an entirely new dimension (time) that the network can exploit. This is not just reformatting — it fundamentally changes what information is available to the network.

5.1 Rate Coding — More Intense = More Spikes

Rate coding is the simplest and most commonly used approach. The idea maps directly from neuroscience: a brighter pixel (higher value) produces more spikes per unit time. Specifically, each pixel value is treated as a probability: a pixel value of 0.8 means there is an 80% chance of generating a spike at each time step. Over many time steps, bright pixels produce dense spike trains and dark pixels produce sparse ones.

Analogy: Imagine each pixel is a popcorn kernel in a microwave. Hotter kernels (higher values) pop more frequently. After running the microwave for 25 seconds (time steps), hot kernels have popped many times and cold kernels have barely popped at all. The pattern of pops encodes the image.

The trade-off: more time steps produce a more accurate encoding but require more computation. For simple tasks like MNIST digit recognition, 25 time steps is often sufficient. For more complex tasks, 50–100 time steps may be needed.

5.2 Latency Coding — Important Stuff Arrives First

In latency coding, information is encoded in when a spike occurs rather than how often. A high-value input fires a spike early in the simulation; a low-value input fires late or not at all. This is like a race: the strongest signals cross the finish line first.

Latency coding is more efficient than rate coding (each neuron fires at most once), and it can enable faster decisions since the network sees the most important information first. However, it can be harder to train because the network must learn to interpret timing differences rather than simple spike counts.

5.3 Delta Modulation — Only React to Change

Delta modulation is the most biologically plausible encoding. Instead of encoding absolute values, it produces spikes only when the input changes by more than a threshold. If nothing changes, no spikes are generated. This is exactly how your retina works — photoreceptor cells respond to changes in light, not to steady illumination.

(This is also why you stop "seeing" something if it is perfectly still on your retina — a phenomenon called neural adaptation.)

Delta modulation produces the sparsest spike trains of all encoding methods, which means the most energy-efficient processing. However, it requires temporal input data (video, sensor streams) rather than static images.

5.4 Direct Input — Let the Network Figure It Out

A practical alternative is to skip encoding entirely and feed raw continuous values directly into the first layer. The first layer of LIF neurons then acts as a learned encoder, converting continuous inputs into spike trains through their natural integrate-and-fire dynamics. This approach is sometimes used when it is unclear which encoding would work best, and it lets the network learn its own optimal encoding during training.

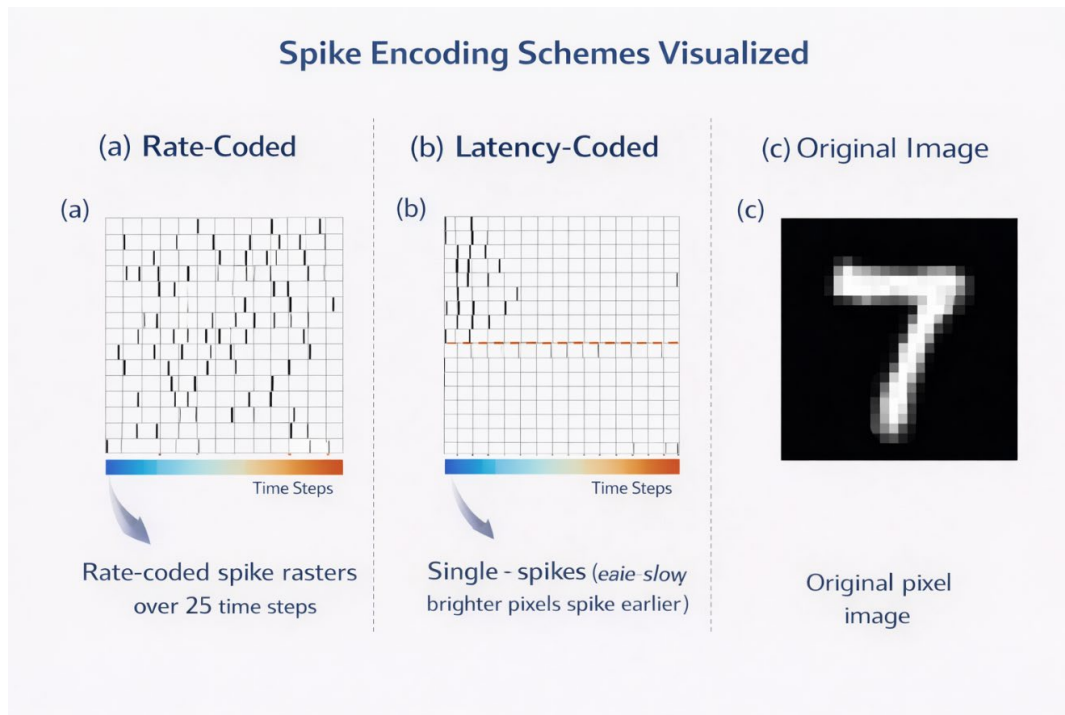
KEY INSIGHT: Encoding Choice Affects Everything Downstream

Your encoding choice is not just a preprocessing detail — it shapes the entire training and inference pipeline:

- Rate coding is the most forgiving (easy to train, works with existing loss functions) but generates the most spikes, reducing efficiency gains.
- Latency coding is the most efficient (fewest spikes) but hardest to train — subtle timing differences can be lost during learning.
- Delta modulation gives the best efficiency on temporal data, but is unusable for static images.
- Direct input offloads the encoding decision to the network itself, trading simplicity for potentially suboptimal spike patterns.

In practice, most researchers start with rate coding (reliable) and optimize to latency or delta modulation once the model works. This mirrors a common engineering pattern: make it work first, then make it efficient.

Encoding Method	How It Works	Analogy	Best For
Rate Coding	Spike probability = input value	Hotter popcorn pops more often	Static images (MNIST, CIFAR)
Latency Coding	High value → early spike	A race: strongest signals arrive first	Fast inference, edge devices
Delta Modulation	Spike only on change	A motion detector: reacts to movement	Video, temporal data, DVS cameras
Direct Input	Feed raw values, let first layer encode	Give ingredients, let chef decide	When encoding choice is unclear



✓ KNOWLEDGE CHECK

Test your understanding before moving on:

1. In rate coding, what determines how many spikes a pixel generates? What happens to a pixel with value 0.0?
2. Why is latency coding more efficient than rate coding in terms of total number of spikes?
3. Give a real-world example of delta modulation in biology (hint: think about what your eyes do).
4. If you were building an SNN to process live video from a drone, which encoding method would you choose and why?
5. In the popcorn analogy for rate coding, what corresponds to 'time steps'? What corresponds to 'spike probability'?

🎉 CONGRATULATIONS — Part 1 Complete!

You now understand the conceptual foundation of spiking neural networks: the three generations of neural networks, why spikes matter for efficiency and temporal processing, how to connect LIF neurons into layered networks, and how to encode data as spikes.

Part 2 tackles the hardest question: how do you TRAIN a network whose neurons communicate with sharp, non-smooth spikes? The answer is one of the most elegant tricks in modern AI. Let's go!

PART 2

Training Spiking Neural Networks

The central challenge: how do you compute gradients through a binary spike?

💡 WHY SHOULD YOU CARE? The Training Problem is THE Problem

If we could not train SNNs with gradient-based methods, they would remain a neuroscience curiosity with no practical AI applications. The surrogate gradient breakthrough (2019) is what unlocked the field for engineers like you. Understanding this technique is the single most important skill for working with SNNs — it is what every neuromorphic computing job posting assumes you know.

Bonus: the concepts here (handling non-differentiable functions, BPTT) appear in many other AI contexts — reinforcement learning, quantization-aware training, and discrete optimization. This knowledge transfers broadly.



SECTION 6

The Training Problem — Why Backpropagation Breaks

This section explains the single biggest challenge in SNN research: how do you train a network whose neurons communicate with binary, non-smooth spikes? To understand the problem, we need to revisit how backpropagation works and see exactly where it fails.

6.1 How Backpropagation Works (Conceptual Review)

You have used backpropagation extensively in your Deep Learning course. Here is the core idea in plain language: after the network makes a prediction, we measure how wrong it was (the loss). Then we ask: "For each weight in the network, if I changed it just a tiny bit, would the prediction get better or worse?" The answer to that question is the gradient, and it tells us which direction to adjust each weight.

To compute gradients, backpropagation uses the chain rule — it traces the path from the loss, backward through each layer, asking at each step: "How sensitive is this layer's output to a change in its input?" This sensitivity is the derivative of the layer's function. If every function along the path is smooth, the chain rule gives us clean, useful gradients. If any function along the path is not smooth, the chain breaks.

6.2 Where It Breaks: The Spike Function

Here is the problem. The spike function says: "If the membrane potential is above the threshold, output 1. Otherwise, output 0." In math, this is called a Heaviside step function — it jumps from 0 to 1 in a single instant, with no smooth transition in between.

When backpropagation reaches this function and asks "How sensitive is the spike output to a tiny change in membrane potential?", the answer is devastating:

If the potential is below threshold: the sensitivity is zero. A tiny change does not flip the spike. The gradient is dead.

If the potential is exactly at threshold: the sensitivity is technically infinite — the tiniest nudge flips the spike. This is numerically useless.

If the potential is above threshold: the sensitivity is again zero. The neuron is already firing; a tiny change does not matter.

The result: the gradient signal dies at every spike function in every neuron at every time step. The network receives no learning signal. It cannot learn.

The Dead Gradient Problem — Explained with a Metaphor

Smooth Slope
(Gradients to guide training)



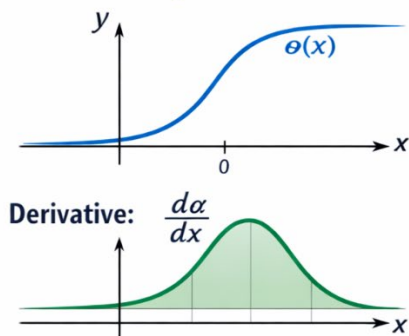
Cliff
(Sudden, abrupt end)



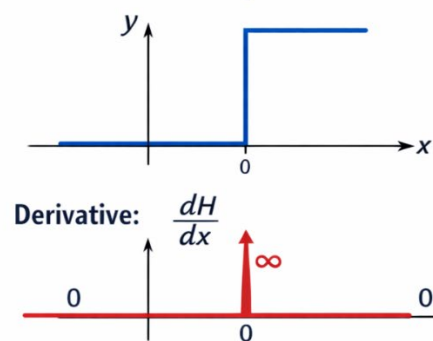
Backpropagation needs the sloping hill. SNNs give us the **cliff**.

The Dead Gradient Problem — Visualized

Smooth Sigmoid Function



Heaviside Step Function



Backpropagation needs the left. SNNs give us the **right**.

FOUNDATION: Connecting to Your Deep Learning Knowledge

Remember the 'dead neuron' problem with ReLU? When a ReLU neuron's input is negative, its output is zero and its gradient is zero — it stops learning. The fix was Leaky ReLU, which lets a small gradient through even for negative inputs.

The spike function has the same type of problem, but much worse. With ReLU, only neurons with negative inputs have zero gradients. With the Heaviside spike function, ALL neurons have zero gradients — whether they are above, below, or far from the threshold. The gradient is dead everywhere.

The solution (Section 7) is conceptually similar to the ReLU→Leaky ReLU fix: replace the problematic derivative with something that works better, while keeping the original function's behavior.

KNOWLEDGE CHECK

Test your understanding before moving on:

1. In your own words, explain why backpropagation requires smooth, differentiable functions.
2. What is the Heaviside step function, and why does its derivative cause problems for learning?
3. How is the dead gradient problem in SNNs similar to — but worse than — the dead neuron problem in ReLU networks?
4. If a neuron's membrane potential is well below the threshold, what gradient does backpropagation receive? Why is this a problem?

SECTION 7

Surrogate Gradients — The Elegant Solution

The surrogate gradient method is the breakthrough that made modern SNN training practical. The idea is surprisingly simple and elegant.

7.1 The Core Trick: Use Two Different Functions

During the **forward pass** (when the network processes data), use the **real Heaviside step function**. The network genuinely produces binary spikes — 0 or 1, nothing in between. This keeps the SNN behaving like a true spiking network.

During the **backward pass** (when computing gradients for learning), **swap in a smooth function** that provides useful, non-zero gradients. The network pretends the spikes were smooth for the purpose of figuring out how to adjust its weights.

That is it. The network behaves like a spiking network but learns as if it were a smooth network. This is why it is called a "surrogate" — the smooth function stands in for the real spike function during learning, like a stunt double in a movie.

💡 KEY INSIGHT: Why This Works

The surrogate gradient works because it preserves the essential learning information: neurons that are CLOSE to the firing threshold should receive the strongest learning signal. Why? Because a small weight adjustment could push them over the threshold (making them fire) or pull them back (stopping them from firing). These are the neurons where learning can have the most impact.

Neurons far below threshold get weak gradients — they would need big changes to start firing.

Neurons far above threshold get weak gradients — they are already firing reliably.

Neurons near the threshold get strong gradients — this is where the action is.

The surrogate function is shaped like a smooth hill centered on the threshold, which naturally provides this behavior.

7.2 Common Surrogate Functions

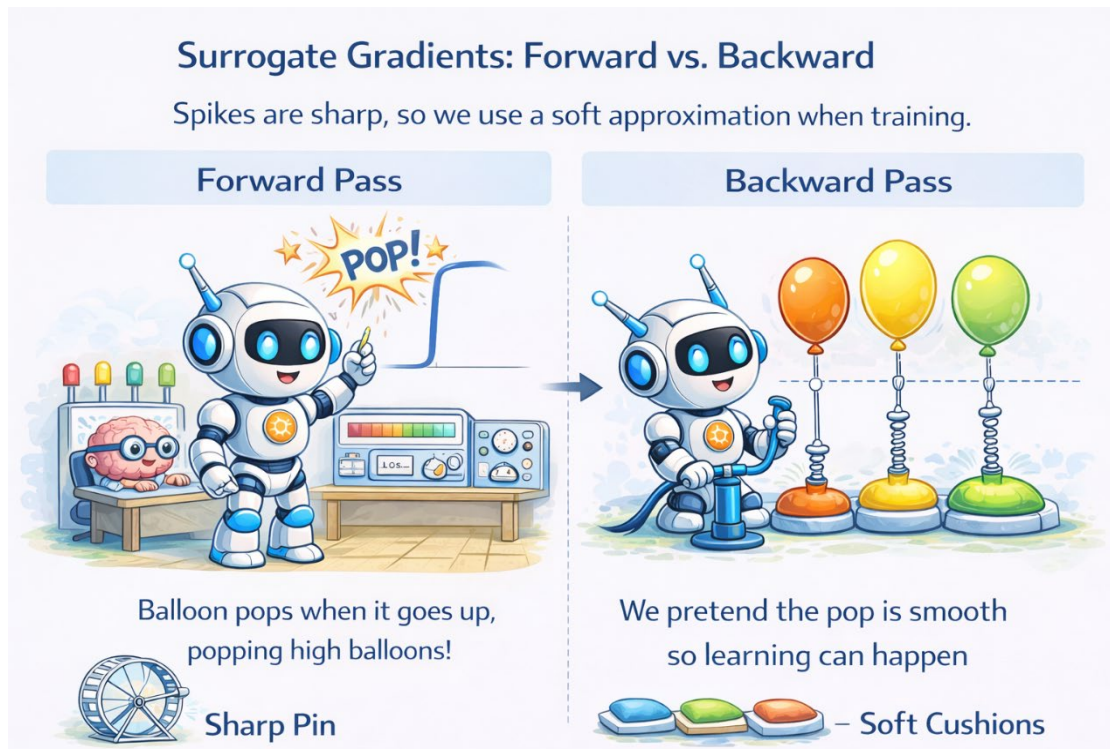
Several smooth functions have been proposed as surrogates. The good news is that SNN training is remarkably robust to which one you choose — they all work reasonably well as long as they are smooth, peak near the threshold, and taper off away from it. Here are the most common:

Fast Sigmoid — The most widely used and the default in `snnTorch`. It produces a smooth bell-shaped gradient centered on the threshold. This is what you will use in the lab.

Arctangent — Similar shape but with slightly wider "tails," meaning neurons further from the threshold still receive some gradient. This can help in deeper networks.

Triangular (Piecewise Linear) — A simple tent-shaped function. Less smooth but very fast to compute, making it attractive for hardware implementations.

Straight-Through Estimator — The crudest approach: just pretend the derivative is 1 everywhere. Surprisingly, this sometimes works for shallow networks, but it provides no useful information about distance from threshold.



REAL-WORLD APPLICATION: Surrogate Gradients in Research

Surrogate gradient methods have become the dominant approach for training SNNs. The foundational paper (Neftci, Mostafa, & Zenke, 2019) has been cited thousands of times. In 2025, Zenke's group published new research providing a deeper theoretical understanding of WHY surrogate gradients work so well — they showed that in networks with stochastic (noisy) neurons, the surrogate derivative is actually related to the mathematical derivative of the neuron's noise function.

Key takeaway: what started as a clever engineering hack now has solid theoretical grounding. This is common in AI — practical methods often work before we fully understand why.

7.3 How This Connects to Biological Learning

It is worth noting that the brain does not use surrogate gradients or backpropagation. Instead, biological neurons adjust their connections using local learning rules — rules that depend only on the activity of the two neurons connected by a synapse, not on a global error signal propagated backward through the entire network.

The most important biological learning rule is Spike-Timing-Dependent Plasticity (STDP), which you will study in depth in Module 06. STDP strengthens a connection when the sending neuron fires just before the receiving neuron (suggesting the input helped cause the output) and weakens it when the order is reversed. This is a local,

biological mechanism that achieves something functionally similar to what surrogate gradients achieve computationally.

Surrogate gradients are an engineering tool that lets us leverage the powerful training infrastructure of deep learning (PyTorch, GPUs, automatic differentiation) for spiking networks. Biology uses a different but related approach. Understanding both gives you a richer picture.

✓ KNOWLEDGE CHECK

Test your understanding before moving on:

1. In your own words, explain the surrogate gradient trick using the 'stunt double' analogy. What is the stunt double, and when does it appear?
2. Why do neurons NEAR the firing threshold receive the strongest surrogate gradients? Why is this desirable for learning?
3. Why is it good news that SNN training is robust to the choice of surrogate function?
4. Name the surrogate function you will use in the lab and what library provides it.
5. How does the brain's approach to learning (STDP) differ from surrogate gradient training? What do they have in common?

SECTION 8

Backpropagation Through Time (BPTT) in SNNs

Surrogate gradients solve the spike function problem, but there is a second challenge: each neuron's membrane potential depends on its own past values. The potential at time step 10 was influenced by inputs at time steps 1 through 9. To train the network properly, we need to account for these temporal dependencies. The algorithm that does this is called Backpropagation Through Time, or BPTT.

8.1 The Concept: Unrolling Through Time

Imagine taking your SNN and making a copy for each time step, then lining up all the copies in a row. Time step 1's network feeds into time step 2's network, which feeds into time step 3's, and so on. The membrane potentials are the threads that connect them — each copy inherits the membrane state from the previous copy.

This "unrolled" view creates one very deep network. If you simulate 25 time steps with 3 layers, the unrolled graph is effectively 75 layers deep. Gradients flow backward through this entire chain — through time and through layers simultaneously. This is exactly how RNNs and LSTMs are trained, which is why the technique shares the same name.

💡 **KEY INSIGHT: Every SNN is a Special Type of RNN**

Here is a conceptual bridge that may help: mathematically, every spiking neural network is a special kind of Recurrent Neural Network. The parallels are direct:

- The membrane potential plays the role of the hidden state (memory that carries forward).
- The decay factor β plays the role of a forget gate (like in an LSTM).
- The spike function plays the role of a gated output.

The key difference: in a standard RNN, the 'activation' is continuous. In an SNN, it is binary (spike or no spike). This binary activation is what enables the energy efficiency advantages on neuromorphic hardware.

Because SNNs are structurally similar to RNNs, they can be trained with the same BPTT algorithm — once we handle the spike function with surrogate gradients.

8.2 The Training Loop — Step by Step

Here is what happens when you train an SNN, in plain language:

- 1. Initialize:** Set all membrane potentials to zero (like emptying all the buckets before starting).
- 2. Loop through time steps:** For each of the T time steps (say, $T=25$), convert the input into spikes for this step, run those spikes through all layers, let neurons integrate, fire, and reset. Record the output spikes at each step.
- 3. Compute the loss:** Count how many times each output neuron spiked across all time steps. Compare these spike counts to the target label. For example, if the image shows a '3,' we want the output neuron labeled '3' to have spiked the most.
- 4. Backpropagate:** Send the error signal backward through the entire unrolled graph — through time and through layers — using surrogate gradients at every spike function.
- 5. Update weights:** Use a standard optimizer (Adam, SGD) to adjust all the weights based on the computed gradients.
- 6. Repeat:** Move to the next training example and do it all again.

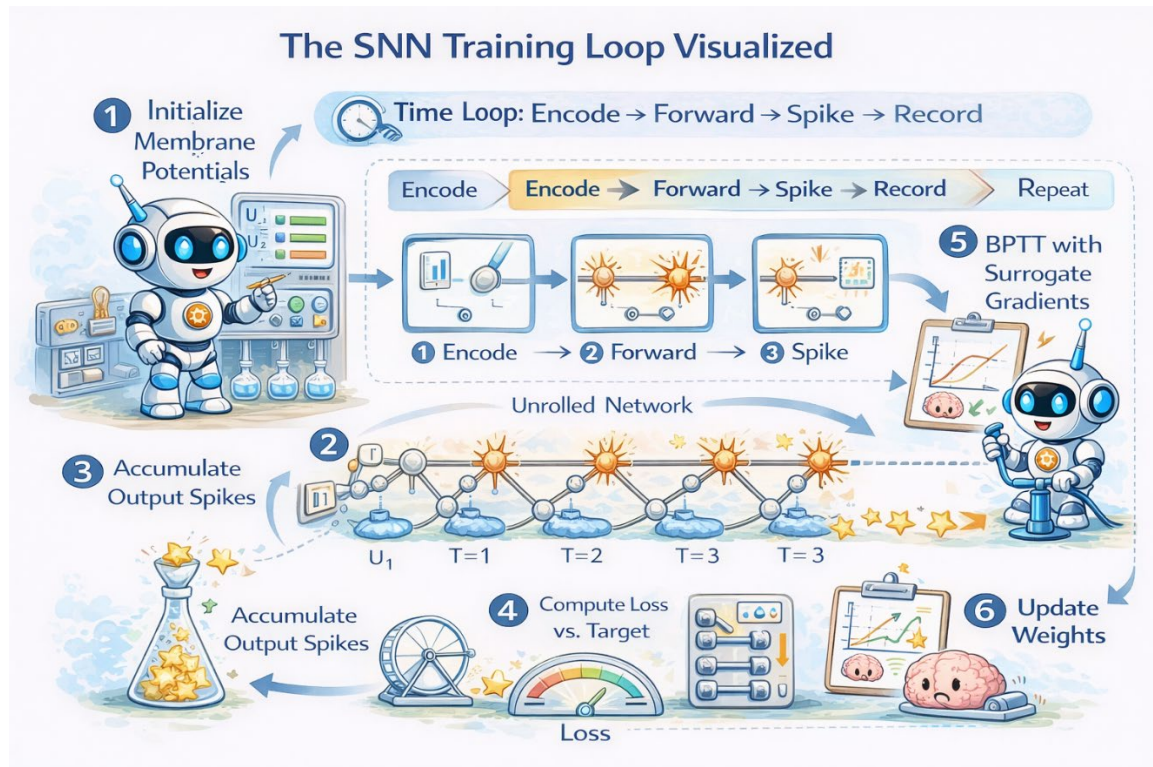
8.3 Interpreting Output Spikes — Making a Decision

After running the network for T time steps, we need to extract a final answer from the output layer's spike trains. There are three main approaches:

Rate decoding: Count total spikes from each output neuron. The neuron with the most spikes wins. This is the most common approach and what you will use in the lab. It is robust and easy to implement.

Latency decoding: The first output neuron to fire wins. This can be faster (you might get an answer at time step 5 instead of waiting for all 25), but it is harder to train and less robust.

Membrane potential decoding: Instead of counting spikes, read the final membrane potential of each output neuron at the last time step. The neuron with the highest voltage wins. This approach is gaining traction in recent research because it captures information from neurons that were close to firing but never crossed the threshold — information that rate and latency decoding throw away. Some studies show it can improve accuracy, especially on harder tasks.



☒ KNOWLEDGE CHECK

Test your understanding before moving on:

1. Explain in plain English what BPTT does. Why is it needed for SNNs specifically?
2. How is training an SNN similar to training an LSTM? What plays the role of the hidden state?
3. If you simulate 25 time steps with 4 layers, how many effective layers does the unrolled graph have?
4. In rate decoding, how do you determine which class the SNN predicts? Why is this straightforward?
5. What advantage does membrane potential decoding have over rate decoding? What information does it capture that spike counts miss?

SECTION 9

snnTorch — Your Tool for Building SNNs

snnTorch is an open-source Python library built on top of PyTorch. It was developed by Jason Eshraghian at UC Santa Cruz and has become one of the most widely used tools for training spiking neural networks. It is what you will use in this module's lab.

9.1 Why snnTorch Over Other Options?

Several frameworks exist for spiking neural networks. Brian2 is designed for computational neuroscience simulations; NEST handles large-scale brain models; Nengo focuses on cognitive architectures. These are powerful but oriented toward neuroscience researchers, not AI engineers.

snnTorch is designed specifically for people with a deep learning background — like you. It extends PyTorch's computational graph with spiking neuron modules, which means everything you already know about PyTorch works identically: DataLoaders, loss functions, optimizers, GPU acceleration, model saving. The only addition is spiking neuron layers and surrogate gradient functions.

9.2 Core Building Blocks

Here are the key components of snnTorch that you will use:

snn.Leaky — The standard LIF neuron module. You create it by specifying the decay factor β and a surrogate gradient function. This is the workhorse neuron for most tasks.

snn.Synaptic — A more biologically realistic neuron with two time constants: one for synaptic current decay and one for membrane potential decay. More complex but sometimes gives better results for temporal tasks.

snntorch.spikegen — Functions for encoding data as spikes. Includes `rate_conv` (rate coding), `latency` (latency coding), and `delta` (delta modulation).

snntorch.surrogate — Surrogate gradient functions: `fast_sigmoid` (default), `atan`, `sigmoid`, and `straight_through_estimator`.

snntorch.functional — Specialized loss functions designed for spike trains, including cross-entropy on spike counts.

9.3 How a snnTorch Model Looks

A snnTorch model is built by alternating PyTorch layers (which handle connections) with snnTorch layers (which handle neuron dynamics). Here is a conceptual view of the convolutional SNN you will build in the lab:

Input image → Convolution (PyTorch) → MaxPool (PyTorch) → LIF neuron (snnTorch) → Convolution (PyTorch) → MaxPool (PyTorch) → LIF neuron (snnTorch) → Flatten

(PyTorch) → Fully connected (PyTorch) → LIF neuron (snnTorch) → Output spike counts

Notice the pattern: PyTorch handles the "wiring" between neurons, and snnTorch handles the "biology" inside neurons. This clean separation makes it easy to experiment with different architectures.

FOUNDATION: What Your Lab Code Will Look Like (Conceptual Preview)

Here is a simplified, conceptual view of the snnTorch model definition you will build. This is NOT the full lab code — just a preview so the lab feels less intimidating:

```
# Define layers (PyTorch handles connections)
conv1 = nn.Conv2d(1, 8, kernel_size=3) # 8 filters
conv2 = nn.Conv2d(8, 16, kernel_size=3) # 16 filters
fc = nn.Linear(16 * 5 * 5, 10)         # 10 output classes

# Define spiking neurons (snnTorch handles biology)
lif1 = snn.Leaky(beta=0.9) # slow leak = longer memory
lif2 = snn.Leaky(beta=0.9)
lif3 = snn.Leaky(beta=0.9)

# Forward pass runs inside a TIME LOOP
for step in range(num_steps): # e.g., 25 steps
    x = conv1(spike_input[step]) # PyTorch: connections
    spike1, mem1 = lif1(x, mem1) # snnTorch: neuron dynamics
    ... (continue through layers)
```

The key insight: each snnTorch neuron returns TWO things — the spike (binary output) AND the membrane potential (internal state). The membrane potential carries forward to the next time step.

REAL-WORLD APPLICATION: From snnTorch to Hardware

snnTorch is not just for learning — it is used in active research at major universities and labs worldwide. The core paper (Eshraghian et al., Proceedings of the IEEE, 2023) is one of the most cited papers in neuromorphic computing.

Critically, snnTorch supports the Neuromorphic Intermediate Representation (NIR) standard, which allows trained models to be exported and deployed on actual neuromorphic hardware like Intel Loihi 2. The library also supports quantization-aware training — reducing the precision of weights and activations for deployment on resource-constrained edge devices.

The skills you build in this lab have a direct path from education to industry deployment.

KNOWLEDGE CHECK

Test your understanding before moving on:

1. What is the division of labor between PyTorch layers and snnTorch layers in a model?
2. Name three components of the snnTorch library and briefly describe what each does.
3. Why is the NIR export feature of snnTorch significant for your career?
4. In the conceptual code preview, why does each snnTorch neuron return TWO values (spike and membrane potential)?

CONGRATULATIONS — Part 2 Complete!

You have conquered the hardest part of this module! You now understand why backpropagation breaks on spikes (dead gradients everywhere), how surrogate gradients solve this with a clever forward/backward split, how BPTT handles temporal dependencies, and how snnTorch makes all of this practical with clean PyTorch integration.

Part 3 is the payoff: the specialized hardware that makes spiking networks shine. Without neuromorphic chips, SNNs are interesting theory. WITH them, they become a game-changing technology. Let's see why.

PART 3

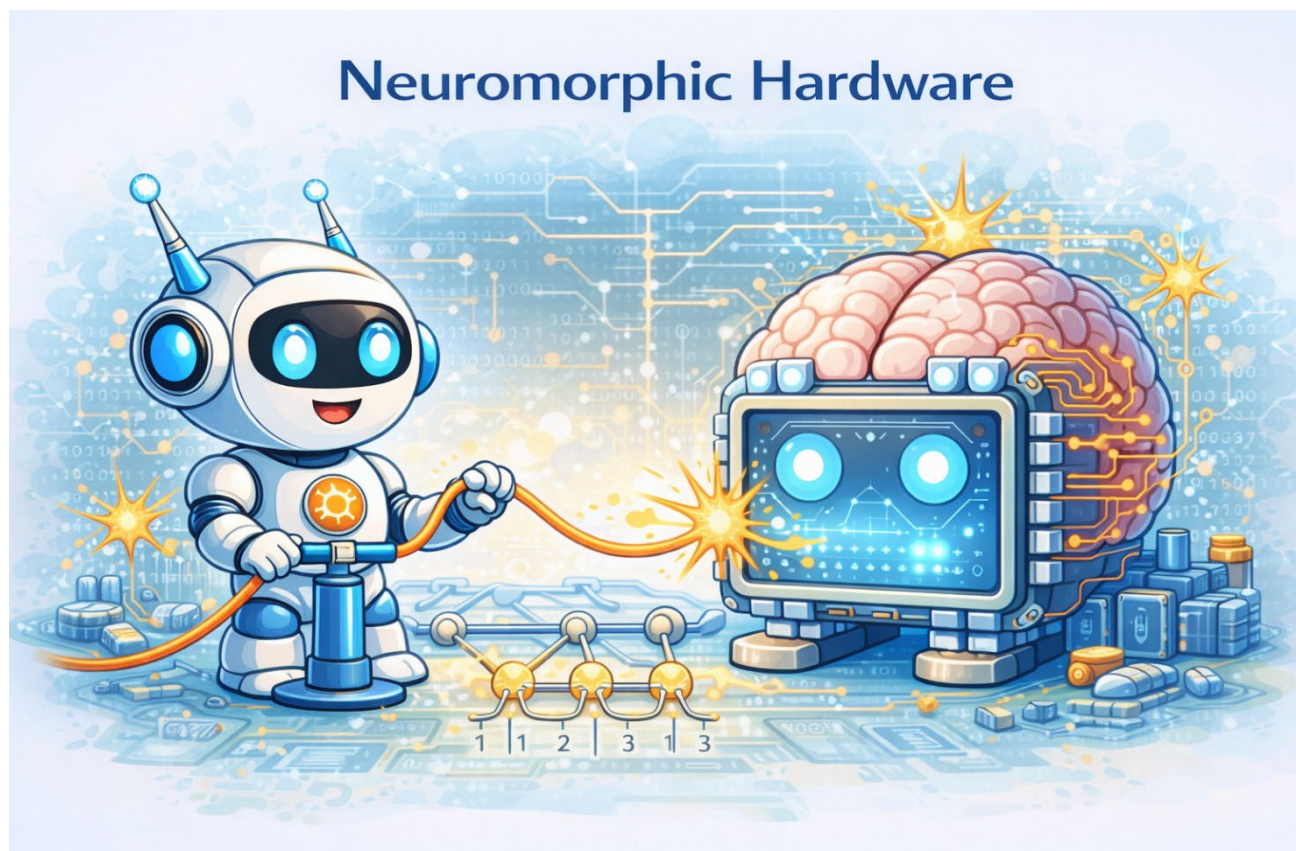
Neuromorphic Hardware

Why spikes only reach their full potential on hardware designed to exploit them.

💡 WHY SHOULD YOU CARE? Hardware is Where the Efficiency Actually Happens

Here is a truth that surprises many students: running an SNN on a GPU is often **SLOWER** and **LESS** efficient than running a conventional ANN on the same GPU. The sparsity and event-driven advantages of SNNs only materialize on hardware specifically designed to exploit them. Without neuromorphic hardware, SNNs are an elegant theory with no practical efficiency advantage.

Understanding neuromorphic hardware is not optional for this field — it is what separates someone who understands SNNs conceptually from someone who can actually deploy them in the real world.



SECTION 10

What Is Neuromorphic Computing?

Running a spiking neural network on a conventional GPU is like driving a sports car in a traffic jam — you have the potential for something great, but the environment does not let you use it. GPUs are designed for dense, synchronous matrix operations. They process every neuron at every time step, regardless of whether a spike occurred. The sparsity and event-driven nature of SNNs — the very properties that make them efficient — are completely wasted on hardware that does not know how to exploit them.

Neuromorphic computing refers to computer hardware designed from the ground up to exploit brain-like computation. The core design principles are:

Event-driven processing: Only active neurons consume energy. If a neuron does not spike, no computation occurs. This is the source of energy savings.

Co-located memory and computation: Each neuron stores its weights locally, right where the computation happens. No need to fetch data from a distant memory bank. This eliminates the biggest energy drain in conventional chips.

Massive parallelism: Many thousands of simple processing cores (each simulating neurons) work simultaneously, rather than a few powerful cores handling everything sequentially.

Asynchronous operation: There is no global clock forcing all neurons to compute at the same time. Each neuron processes its inputs whenever they arrive, just like biological neurons.

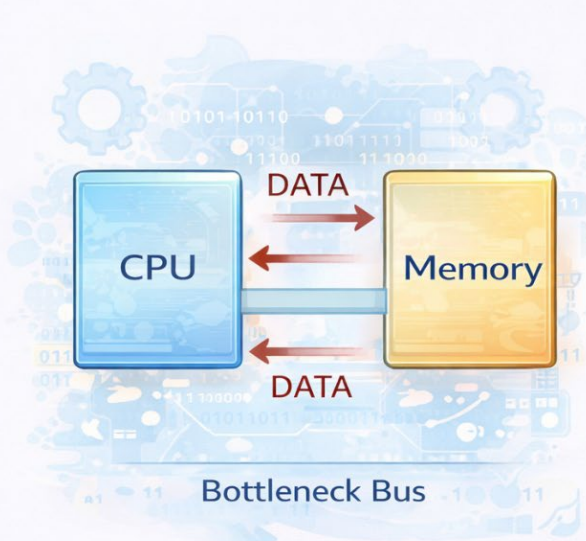
10.1 The Von Neumann Bottleneck

To understand why neuromorphic hardware matters, you need to understand the biggest problem in conventional computing: the von Neumann bottleneck.

In a standard computer — including GPUs — the processor and memory are separate components connected by a data bus. Every computation requires: fetch data from memory → process it → write results back to memory. This constant shuttling of data back and forth is slow and energy-hungry. In neural networks, where billions of weights must be repeatedly fetched from memory for each forward pass, this bottleneck becomes a critical limitation.

The brain has no such bottleneck. Each synapse (connection between neurons) stores its own weight right where the computation happens. There is no separation between memory and processing. Neuromorphic chips attempt to replicate this elegant design, with each processing core containing its own local memory for weights and neuron states.

Von Neumann Architecture vs. Neuromorphic Architecture



Von Neumann Architecture



Neuromorphic Architecture

Processing + Local Memory

🧠➡️🤖 AI CONNECTION: Processing-in-Memory — The Same Idea in Conventional AI

The brain's solution to the memory-processor separation problem is so compelling that even conventional AI hardware is trying to copy it. Processing-in-Memory (PIM) and Compute-in-Memory (CIM) are emerging approaches where computation happens directly inside memory chips, reducing data movement.

Companies like Samsung, SK Hynix, and UPMEM are developing PIM-enabled DRAM chips for AI workloads. Analog AI chips from IBM and Mythic perform matrix multiplications directly in resistive memory arrays — mimicking how neuromorphic synapses store and compute simultaneously.

The takeaway: the neuromorphic computing principles you are learning here are not an isolated niche. The entire semiconductor industry is moving toward brain-inspired architectures, just at different speeds and scales. Understanding these principles positions you at the leading edge of hardware-aware AI.

☑ KNOWLEDGE CHECK

Test your understanding before moving on:

1. What is the von Neumann bottleneck? Use an everyday analogy to explain it.
2. Name the four core design principles of neuromorphic computing.
3. Why does running an SNN on a GPU waste the SNN's main advantages?
4. How does the brain solve the memory-processor separation problem? How do neuromorphic chips copy this solution?

SECTION 11**Intel Loihi 2 and the Hala Point System**

Intel's Loihi 2 is the most prominent neuromorphic research chip today and serves as an excellent case study for understanding how this hardware works in practice.

11.1 Loihi 2 Architecture

A single Loihi 2 chip contains 128 neuromorphic cores. Each core can simulate spiking neurons with programmable dynamics — meaning researchers can define custom neuron models, not just the basic LIF. The chip supports up to 1 million neurons and 120 million synapses on a single piece of silicon roughly the size of your fingernail.

Communication between neurons happens through spike messages sent over an asynchronous network-on-chip. There is no global clock — each core processes spikes as they arrive, just like biological neurons. This asynchronous design is one of the key reasons for Loihi 2's efficiency.

A major upgrade from the original Loihi: Loihi 2 has **fully programmable neuron cores**. The original Loihi only supported a fixed LIF neuron model. Loihi 2 allows researchers to write custom neuron behaviors using microcode instructions — enabling it to emulate many different neuron types, implement on-chip learning rules (including STDP), and even approximate some non-spiking algorithms.

11.2 Hala Point — Scaling to a Billion Neurons

In 2024, Intel announced Hala Point, the world's largest neuromorphic system. Here are the key numbers:

Specification	Hala Point Details
Loihi 2 chips	1,152 chips packaged together
Total neurons	1.15 billion
Total synapses	128 billion
Neuromorphic cores	140,544

Physical size	Six-rack-unit chassis (about the size of a microwave oven)
Maximum power	2,600 watts
Efficiency	Up to 15 TOPS/W (tera-operations per second per watt)
Location	Deployed at Sandia National Laboratories
Comparison	A GPU cluster with 1 billion+ neurons would consume 100,000+ watts

To put this in perspective: Hala Point simulates more neurons than a honeybee brain (about 1 million) and approaches the scale of a mouse brain cortex (about 14 million cortical neurons, though with far more synapses). It does this in a box the size of a microwave, consuming roughly the power of a hair dryer. A GPU-based system with comparable neural capacity would fill a room and require dedicated power infrastructure.

Intel Loihi 2 & Hala Point Neuromorphic System

Loihi 2 Chip



- ~ 1M neurons / chip
- Event-driven spiking cores

Designed for energy-efficient
SNN workloads

Hala Point System



- 1.15B neurons total
- 2,600W system
- Research-scale neuromorphic machine

Intel Neuromorphic System

**INDUSTRY CONNECTION: The Software Bridge — Lava and NIR**

Intel provides Lava, an open-source software framework for programming Loihi 2. Lava lets you describe neural networks in Python and compile them to run on either conventional CPUs or directly on Loihi 2 hardware.

The critical connection to your lab work: models trained in `snnTorch` can be exported via NIR (Neuromorphic Intermediate Representation) and deployed on Loihi 2 through Lava. NIR is like an 'ONNX for spiking networks' — a common format that different SNN tools and hardware platforms can all read.

This means the exact model you train in this module's lab could, in principle, be deployed on actual neuromorphic hardware.

SECTION 12**The Broader Neuromorphic Ecosystem**

Intel Loihi 2 is not the only player in neuromorphic computing. Understanding the broader landscape gives you a sense of where the field is heading and what career opportunities exist.

12.1 Other Neuromorphic Platforms

Platform	Developer	Key Features	Status
Loihi 2 / Hala Point	Intel	Programmable neurons, 1M/chip, Lava framework, NIR support	Research (INRC partners)
TrueNorth	IBM	1M neurons/chip, ultra-low power (70mW), fixed neuron model	Research / Legacy
SpiNNaker 2	U. of Manchester	ARM cores emulating neurons, large-scale brain simulation	Research
BrainScaleS 2	U. of Heidelberg	Analog circuits, 1000× faster than biology, on-chip learning	Research (EU Human Brain Project)
Akida	BrainChip	First commercial neuromorphic chip, edge AI focus	Commercial — Available for purchase
Speck	SynSense	Event-driven vision processor, sub-milliwatt power, pairs with DVS cameras	Commercial



INDUSTRY CONNECTION: Career Opportunities

BrainChip's Akida is particularly noteworthy because it is a commercially available neuromorphic chip — you can actually buy development boards. This means companies can build products today using neuromorphic technology, not just research prototypes.

Organizations hiring neuromorphic engineers include: Intel (INRC program), BrainChip, SynSense, Qualcomm (neuromorphic division), Samsung (neuromorphic research), various defense contractors, and academic research labs. The skill combination they seek: SNN theory + practical framework experience (snnTorch/Lava) + understanding of hardware constraints.

12.2 Event-Driven Cameras (Dynamic Vision Sensors)

One of the most exciting applications of neuromorphic computing is paired with a new type of camera called a **Dynamic Vision Sensor (DVS)**, also known as an event camera. Unlike a conventional camera that captures full frames at a fixed rate (30 or 60 FPS), a DVS has pixels that independently fire events only when they detect a change in brightness — exactly like biological retinal cells.

Instead of outputting frames, a DVS outputs an asynchronous stream of events: (x-coordinate, y-coordinate, timestamp, polarity — brighter or darker). The advantages are remarkable:

Temporal resolution: Microsecond precision, compared to milliseconds for conventional cameras. A DVS can track a bullet in flight.

No motion blur: Because each pixel responds independently and instantly, fast-moving objects have crisp edges.

Extreme dynamic range: 120 dB versus 60 dB for conventional cameras. A DVS can see in both bright sunlight and deep shadow simultaneously.

Very low power: In a still scene, no events are generated, so no power is consumed. Energy use scales with the amount of visual change.

DVS cameras are a natural input source for SNNs: both are event-driven, both represent information through temporal events, and both achieve efficiency through sparsity. The combination of DVS + SNN + neuromorphic chip creates a fully event-driven processing pipeline from sensor to decision.

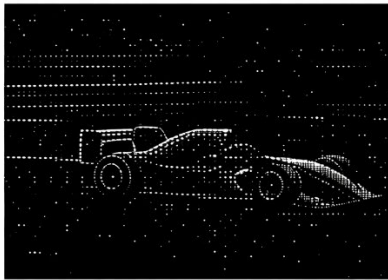
DVS vs. Conventional Camera Output

Conventional Camera Frame



Full image, motion blur

DVS Event Stream



Sparse dots showing
brightness changes

Backpropagation needs the sloping hill. SNNs give us the **cliff**.

12.3 Real-World Applications Already in Use

Neuromorphic computing and SNNs are not theoretical — they are being applied today in several domains:

Always-on keyword spotting: Voice-activated devices ("Hey Siri," "OK Google") that listen continuously for a wake word. Loihi 2 has demonstrated up to 200× lower energy consumption than embedded GPUs for this task — consuming as little as 30 microwatts during standby versus 6+ milliwatts for a comparable embedded GPU solution. This directly translates to longer battery life in wearable devices.

Robotic control: The low latency of neuromorphic processing enables faster sensorimotor loops for robots. Researchers at the National University of Singapore have built neuromorphic robotic skin using Loihi that can detect and respond to touch in real time.

Continual learning: A major 2025 research result demonstrated CLP-SNN (Continually Learning Prototypes on a Spiking Neural Network) running on Loihi 2. The system learned to recognize new objects incrementally without forgetting old ones — solving the catastrophic forgetting problem that plagues conventional deep learning. You will study catastrophic forgetting in Module 06.

Optimization and planning: Neuromorphic systems have shown over 1,000× improvement in energy-delay product for solving certain combinatorial optimization problems (like scheduling or routing) by mapping problem constraints onto spiking network dynamics.

Telecommunications: Ericsson Research is developing neuromorphic AI models for optimizing telecom network architecture, leveraging the real-time processing capabilities and energy efficiency of Loihi 2.

TRY THIS: Design a Neuromorphic System

Imagine you are tasked with building an AI system for a medical wearable that monitors heart rhythms 24/7 and alerts the wearer to dangerous arrhythmias. The device runs on a tiny battery that must last at least one week.

Consider the full pipeline:

- What type of sensor would you use? (Hint: the signal is temporal)
- What encoding method makes the most sense for heart rhythm data?
- Why would an SNN be better than a conventional ANN for this task?
- Which neuromorphic chip would you choose, and why?

There is no single right answer — but this exercise forces you to connect everything from this module into a coherent system design.

KNOWLEDGE CHECK

Test your understanding before moving on:

1. What is the Hala Point system, and how does its power consumption compare to a conventional GPU cluster of similar neural capacity?
2. Name one commercially available neuromorphic chip and explain why its commercial availability matters.
3. How does a Dynamic Vision Sensor work? Why is it called 'event-driven'?
4. Give three advantages of a DVS over a conventional frame-based camera.
5. Why are DVS cameras and SNNs described as 'natural partners'?
6. Describe the continual learning result on Loihi 2. Why is this significant for AI?

SECTION 13

Lab Preview — Training an SNN on MNIST with snnTorch

In this module's lab, you will put everything from this booklet into practice by training a real spiking neural network to classify handwritten digits.

13.1 What You Will Build

You will construct a convolutional spiking neural network (CSNN) — a spiking version of the CNNs you have built in your Computer Vision course. The architecture:

Input (28×28 MNIST images) → Convolutional layer (8 filters) → MaxPool → LIF neurons → Convolutional layer (16 filters) → MaxPool → LIF neurons → Flatten → Fully connected layer (10 outputs) → LIF neurons → Output spike counts

The network uses rate-coded input (each pixel's brightness determines spike probability), the fast sigmoid surrogate gradient, and rate decoding at the output (the output neuron with the most spikes wins). You will simulate the network over 25 time steps per image.

13.2 Tools and Setup

The lab runs in Google Colab, so no local installation is needed. You will use PyTorch (tensor operations and autograd), snnTorch (spiking neuron layers and surrogate gradients), torchvision (MNIST dataset), and matplotlib (visualization). A pre-built Jupyter notebook with detailed instructions will be provided on Canvas.

13.3 What You Will See and Learn

A well-trained spiking CNN can achieve 95–99% accuracy on MNIST — comparable to conventional CNNs. But the training process looks different. You will:

Visualize membrane potential traces — watching neurons integrate inputs and fire spikes over time, connecting the LIF theory from Module 03 to real computational behavior.

Create spike raster plots — seeing which neurons fire at which time steps, revealing the temporal dynamics of the network.

Experiment with the decay factor β — seeing how different values change how quickly neurons "forget" past inputs.

Observe spike count accumulation — watching output neurons gradually accumulate evidence for their class over time steps.

INDUSTRY CONNECTION: From Lab to Deployment Pipeline

The CSNN architecture you build in this lab is not just an exercise. It is similar to architectures being deployed commercially for edge AI vision tasks. After training in `snnTorch`, this model could be:

1. Exported via NIR (Neuromorphic Intermediate Representation)
2. Compiled with Intel's Lava framework
3. Deployed on a Loihi 2 chip or BrainChip Akida for inference at a fraction of the power cost of a GPU

Companies hiring for neuromorphic roles look for exactly this skill set: understanding SNN theory, practical experience with `snnTorch`, and knowledge of hardware deployment constraints.

SECTION 14

Key Takeaways & Bridge to Module 05

14.1 What We Learned

This module took you from the single LIF neuron of Module 03 to complete spiking neural networks that can learn from data and run on specialized hardware. The key takeaways are:

SNNs are fundamentally different: They are not just ANNs with a different activation function. Time is a core computational variable. Neurons maintain state, communicate through discrete spikes, and process information as it unfolds over time.

Surrogate gradients solved the training problem: By using a smooth substitute during the backward pass while keeping real spikes during the forward pass, we can train SNNs with the same tools we use for standard deep learning.

Hardware matters: The efficiency advantages of SNNs only materialize on hardware designed to exploit event-driven, sparse computation. Neuromorphic chips like Loihi 2 eliminate the von Neumann bottleneck and process only active neurons.

The ecosystem is maturing: Commercial chips (Akida), event cameras (DVS), software tools (`snnTorch`, Lava, NIR), and real applications (keyword spotting, robotics, continual learning) are creating a viable pipeline from research to products.

**BRIDGE TO MODULE 05: Sensory Processing & Perception**

Module 04 gave you: NETWORKS (connecting LIF neurons), TRAINING (surrogate gradients, BPTT), HARDWARE (neuromorphic chips).

Module 05 will explore how the brain's visual system processes information:

- The visual cortex hierarchy: $V1 \rightarrow V2 \rightarrow V4 \rightarrow IT$ (from edges to objects)
- How this hierarchy directly inspired CNN architectures you already know
- Event-based vision with silicon retinas (connecting DVS cameras from this module to visual processing)
- Multimodal sensory integration (combining vision, hearing, and touch)

You will see the direct line from Hubel and Wiesel's 1960s experiments on cat visual cortex to the convolutional neural networks powering modern computer vision. And you will explore how event-driven cameras take this connection even further.

SECTION 15**Comprehensive Review Questions**

These questions cover all major topics in this module. Use them for study and exam preparation.

1. Explain the three generations of neural networks in your own words. What makes the third generation fundamentally different from the second?
2. Why do spiking neural networks offer energy efficiency advantages? What specific property of spiking neurons enables this?
3. Using the 'leaky bucket' analogy, describe what happens in a LIF neuron from the moment an input arrives to the moment a spike is emitted.
4. Describe the dead gradient problem in SNN training. Why does the Heaviside step function make standard backpropagation impossible?
5. Explain how surrogate gradients solve the dead gradient problem. Use the 'stunt double' analogy.
6. Why do neurons near the firing threshold receive the strongest surrogate gradients? Why is this desirable for learning?
7. Compare rate coding, latency coding, and delta modulation. Give one scenario where each would be the best choice.
8. Explain what BPTT does and why it is needed. How is it similar to training an LSTM?
9. What is the von Neumann bottleneck? How do neuromorphic chips address it?

- 10.** Describe Intel's Hala Point system: how many neurons it supports, how much power it consumes, and why this matters.
- 11.** How does a Dynamic Vision Sensor differ from a conventional camera? Give three specific advantages.
- 12.** Why are DVS cameras and SNNs described as 'natural partners'?
- 13.** Explain the role of snnTorch in SNN training. What does it add to PyTorch?
- 14.** What is NIR and why is it important for the neuromorphic ecosystem?
- 15.** Describe one real-world application of neuromorphic computing that is already being used today.
- 16.** How does biological learning (STDP) differ from surrogate gradient training? What do they have in common functionally?

SUPER QUICK SUMMARY — Module 04 in 60 Seconds

SNNs are the 3rd generation of neural networks — neurons communicate with discrete spikes over time, not continuous values.

Why spikes? Energy efficiency (only active neurons compute), natural temporal processing (time is built in), ultra-low latency (respond before full processing), biological plausibility.

Encoding converts data to spikes: rate coding (more intense = more spikes), latency coding (important = early spike), delta modulation (only changes trigger spikes).

Training challenge: spike function (Heaviside) has zero gradient everywhere → backpropagation breaks.

Solution: surrogate gradients — use real spikes in forward pass, smooth substitute in backward pass (like a stunt double). Train with BPTT (same as RNNs).

Tool: snnTorch extends PyTorch with spiking neurons. PyTorch handles connections, snnTorch handles neuron biology.

Hardware: neuromorphic chips (Loihi 2, Akida) eliminate the von Neumann bottleneck with event-driven, co-located memory/computation. Hala Point: 1.15B neurons, 2,600W vs 100,000+W for equivalent GPU cluster.

DVS cameras + SNNs + neuromorphic chips = fully event-driven AI pipeline from sensor to decision.

SECTION 16

Glossary

Asynchronous Processing: Computing without a global clock — each unit processes events as they arrive, independent of other units.

Backpropagation Through Time (BPTT): An algorithm for training networks with temporal dynamics (RNNs, SNNs) by 'unrolling' the network across time steps and propagating gradients backward through both layers and time.

Beta (β) / Decay Factor: A number between 0 and 1 that controls how quickly the LIF neuron's membrane potential leaks toward rest. Higher β = slower leak = longer memory.

Delta Modulation: A spike encoding method that generates spikes only when the input changes by more than a threshold — inspired by how biological retinal cells work.

Dynamic Vision Sensor (DVS): An event-driven camera where each pixel independently fires an event when it detects a brightness change, producing microsecond-resolution temporal data.

Event-Driven Processing: A paradigm where computation occurs only when events (spikes) happen — no events means no computation and no energy consumed.

Hala Point: Intel's large-scale neuromorphic system: 1,152 Loihi 2 chips, 1.15 billion neurons, 128 billion synapses, deployed at Sandia National Laboratories.

Heaviside Step Function: The mathematical function that outputs 0 for inputs below a threshold and 1 above. Describes spike firing in SNNs. Its derivative is zero almost everywhere, causing the dead gradient problem.

Latency Coding: A spike encoding method where information is represented by the timing of a spike — high values produce early spikes, low values produce late spikes.

Lava: Intel's open-source software framework for programming Loihi 2 neuromorphic processors.

Leaky Integrate-and-Fire (LIF): A simplified neuron model where inputs accumulate with exponential decay, and a spike fires when the potential crosses a threshold. The 'leaky bucket' of neuroscience.

Loihi 2: Intel's second-generation neuromorphic research chip: 128 programmable cores, up to 1 million neurons per chip, asynchronous processing.

Membrane Potential Decoding: An output interpretation method that reads the final membrane potential of output neurons rather than counting spikes, capturing sub-threshold information.

Neuromorphic Computing: Hardware architectures inspired by the brain: co-located memory/computation, event-driven processing, massive parallelism, asynchronous operation.

Neuromorphic Intermediate Representation (NIR): A standardized format for transferring SNN models between different software frameworks and hardware platforms — like 'ONNX for spiking networks.'

Processing-in-Memory (PIM): An emerging hardware approach where computation happens directly inside memory chips, reducing data movement — inspired by the same brain principles as neuromorphic computing.

Rate Coding: A spike encoding method where higher input values produce higher spike frequencies — the most common encoding for static images.

snnTorch: An open-source Python library extending PyTorch with spiking neuron models and surrogate gradient functions for training SNNs.

Spiking Neural Network (SNN): A neural network where neurons communicate through discrete temporal events (spikes) and maintain internal dynamical state. The 'third generation' of neural networks.

Surrogate Gradient: A training technique that replaces the non-differentiable spike derivative with a smooth function during the backward pass, enabling gradient-based SNN training.

Von Neumann Bottleneck: The performance and energy limitation caused by separating memory and processing in conventional computers, requiring constant data transfer between them.

SECTION 17

References & Recommended Resources

Core Reading

Eshraghian, J.K., et al. (2023). Training Spiking Neural Networks Using Lessons From Deep Learning. *Proceedings of the IEEE*, 111(9), 1016–1054.

Neftci, E.O., Mostafa, H., & Zenke, F. (2019). Surrogate Gradient Learning in Spiking Neural Networks. *IEEE Signal Processing Magazine*, 36(6), 51–63.

Hawkins, J. (2021). *A Thousand Brains: A New Theory of Intelligence*. Basic Books.

Carter, R. (2019). *The Human Brain Book*. DK Publishing.

Online Resources

snnTorch Documentation: snntorch.readthedocs.io

snnTorch Tutorials (Colab Notebooks): github.com/snntorch/Spiking-Neural-Networks-Tutorials

Intel Neuromorphic Research: intel.com/content/www/us/en/research/neuromorphic-computing.html

Open Neuromorphic Community: open-neuromorphic.org

Intel Lava Framework: lava-nc.org

Neuronal Dynamics (free online textbook): neurondynamics.epfl.ch

Key Papers

Davies, M., et al. (2021). Advancing Neuromorphic Computing with Loihi: A Survey of Results and Outlook. *Proceedings of the IEEE*.

Gygax, J. & Zenke, F. (2025). Elucidating the Theoretical Underpinnings of Surrogate Gradient Learning. *Neural Computation*, 37(5).

CLP-SNN (2025). Real-time Continual Learning on Intel Loihi 2. [arXiv:2511.01553](https://arxiv.org/abs/2511.01553).

— *End of Module 04* —

Next: Module 05 — Sensory Processing & Perception